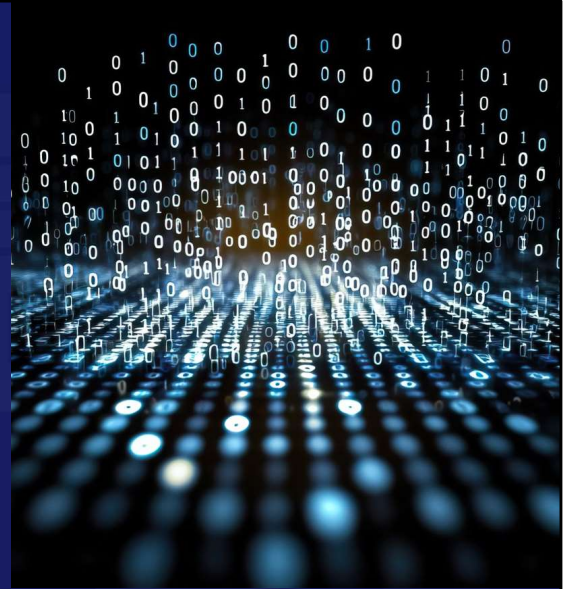


Object Oriented and Design

Presented by Dr Owen Noel Newton Fernando

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted



In this lecture, we begin by understanding what Object-Oriented Programming, or OOP, actually means. OOP is a programming paradigm that structures software around objects rather than just functions and logic. These objects represent real-world entities, allowing us to model problems in a way that is intuitive and closer to how we think naturally.

Each object in OOP combines data and behaviour. The data is stored as attributes, while the behaviour is defined using methods. This approach helps us organise code more clearly, making it easier to understand, extend, and maintain as systems grow in complexity.

Throughout this session, we will explore how Python supports OOP concepts such as classes, objects, and methods, and how these ideas form the foundation for writing modular, reusable, and well-structured programs.

Image source: https://www.freepik.com/premium-ai-image/binary-code-background-made-zeros-ones-representing-essence-digital-technology_383619980.htm



Objectives

- Introduce the core concepts and principles of Object-Oriented Programming (OOP) in Python.
- Explain how objects and classes model real-world entities and behaviours.
- Show how to define and use attributes, methods, and constructors.
- Demonstrate key OOP concepts such as inheritance, polymorphism, encapsulation, and abstraction.
- Promote efficient code organisation, reusability, and modular design.



© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

The main focus here is to build a clear understanding of the core principles of Object-Oriented Programming in Python. Rather than writing isolated pieces of code, the aim is to structure programs around objects that combine data and behaviour in a meaningful way.

Attention is then placed on how classes and objects can be used to model real-world entities. You will learn how to define attributes, methods, and constructors, and how these components work together to represent state and behaviour within a program.

Finally, the session covers key OOP concepts such as inheritance, polymorphism, encapsulation, and abstraction. These ideas support better code organisation, encourage reuse, and help you design modular solutions that are easier to extend and maintain over time.

Image Source: https://www.freepik.com/premium-photo/worldwide-server-data_260897651.htm

Learning Outcome



At the end of this lecture, students will be able to:

- Define objects, classes, and key OOP terms in Python.
- Create and manage class instances and attributes.
- Implement methods and constructors to handle object data.
- Apply inheritance and polymorphism to extend class behaviour.
- Use encapsulation and abstraction to organise and protect data.
- Design simple object-oriented programs that are clear and reusable.



© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

By the end of this session, you should be comfortable with the fundamental terminology used in Object-Oriented Programming, including objects, classes, attributes, and methods. These terms form the language we use to reason about object-based program design in Python.

You will also develop the ability to create and manage class instances, initialise object data using constructors, and implement methods that operate on that data. This practical understanding allows you to control how objects behave and interact within a program.

In addition, you will apply inheritance and polymorphism to extend existing classes, while using encapsulation and abstraction to organise and protect data. Together, these skills enable you to design simple yet well-structured object-oriented programs that are clear, reusable, and maintainable.

Contents



- Introduction to Object-Oriented Programming (OOP).
- OOP Concepts: Exploring inheritance, polymorphism, encapsulation, and abstraction.
- Writing and testing class-based programs in Python.
- Summary and key takeaways from OOP concepts.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted



The session is organised to gradually build your understanding of Object-Oriented Programming. We begin with a general introduction to OOP, focusing on why this approach is widely used in modern software development and how it differs from purely procedural programming.

The discussion then moves into the core OOP concepts, including inheritance, polymorphism, encapsulation, and abstraction. These concepts are explored through examples in Python, helping you see not just the theory, but how each idea is applied in actual code.

Finally, the session emphasises writing and testing class-based programs. By the end, the aim is not only to understand OOP terminology, but also to confidently apply these concepts when designing and implementing well-structured Python programs.

Image Source: https://www.freepik.com/premium-photo/worldwide-server-data_260897651.htm

Introduction to Object-Oriented Programming

Introduction, Objects , Classes, attributes, and methods.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Object-Oriented Programming is a way of structuring programs by focusing on objects rather than isolated functions or variables. Each object represents a meaningful entity that combines data with the operations that act on that data, making programs easier to reason about and manage.

This approach allows complex systems to be broken down into smaller, self-contained components. Instead of dealing with scattered data, related information and behaviour are grouped together, which improves clarity and reduces the risk of errors as programs grow.

Python supports Object-Oriented Programming in a flexible and intuitive manner. Through the use of classes and objects, it becomes possible to model real-world problems more naturally, leading to code that is easier to extend, reuse, and maintain over time.

Image Source: <https://www.istockphoto.com/vector/binary-tree-gm479524481-17312880?clarity=false>

Create A Customized Data Type Class

- Limited built-in data types in Python (any programming languages)
 - bool, int, float, complex (3+4j).

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Python provides a small set of built-in data types such as integers, floating-point numbers, booleans, and complex numbers. While these types are sufficient for simple programs, they quickly become limiting when we need to represent richer, real-world information.

Create A Customized Data Type Class

- Limited built-in data types in Python (any programming languages)

- bool, int, float, complex (3+4j)..

- In practice, the data can be more complex

- Student Information

- Name – string

- Age – int

- CGPA – float (2 decimal places)

- Etc.

```
class Student:
    def __init__(self, name, age, cgpa):
        self.name = name
        self.age = age
        self.cgpa = cgpa
```

Built-in data types such as integers and floating-point numbers are useful, but they are often insufficient for representing structured information. In real applications, data typically consists of multiple related values that need to be treated as a single unit rather than as separate variables.

The example shown here demonstrates how student information can be modelled using a custom class. Instead of handling a name, age, and CGPA independently, these attributes are grouped together within a Student class. The constructor, defined using the `__init__` method, ensures that every student object is created with a consistent set of data.

This approach improves readability and reliability. Each student becomes a well-defined object, making the program easier to extend, validate, and maintain as additional attributes or behaviours are introduced.

Create A Customized Data Type Class

- Limited built-in data types in Python (any programming languages)
 - bool, int, float, complex (3+4j)..
- In practice, the data can be more complex
 - Student Information
 - Name – string
 - Age – int
 - CGPA – float (2 decimal places)
 - Etc.
- Benefits of Custom Data Types
 - **Encapsulation**: Bundle data and methods within a single unit.
 - **Reusability**: Create reusable code components.
 - **Abstraction**: Hide implementation details and expose only necessary operations.

```
class Student:
    def __init__(self, name, age, cgpa):
        self.name = name
        self.age = age
        self.cgpa = cgpa
```

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

One key benefit of using custom data types is encapsulation. By defining a class, related data and the methods that operate on that data are bundled together within a single unit. This reduces accidental misuse of data and helps ensure that objects are always used in a controlled and consistent manner.

Another important advantage is reusability. Once a class is defined, it can be reused across different parts of a program or even across multiple projects. This avoids duplication, improves consistency, and significantly reduces development and maintenance effort over time.

Abstraction further enhances code clarity by hiding unnecessary implementation details. Users of a class only need to understand what operations are available, not how they are implemented internally. This makes programs easier to understand, modify, and extend without affecting existing functionality.

Object-Oriented Programming

What is OOP?

- A programming paradigm based on the concept of "**objects**" that encapsulate data and behavior.
- OOPs is a way of organizing code that uses **objects** and **classes** to represent real-world entities and their behavior
- In OOPs, object has attributes that have specific data and can perform certain actions using methods.

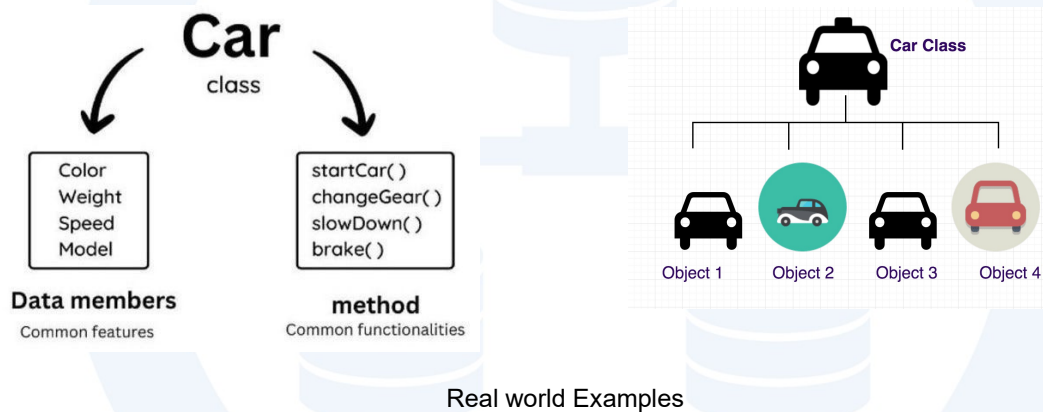


Object-Oriented Programming is a programming paradigm that organises software around objects. Each object encapsulates both data and behaviour, meaning it stores information and also defines what actions can be performed on that information. This structure mirrors how we naturally think about real-world entities.

Rather than writing code as a sequence of instructions, OOP encourages us to model systems using classes and objects. Classes act as blueprints, while objects represent concrete instances that interact with one another. This makes complex programs easier to design, understand, and reason about.

In OOP, each object contains attributes that represent its state and methods that define its behaviour. This clear separation of responsibility leads to cleaner code, better organisation, and improved maintainability as software systems evolve.

Object-Oriented Programming



© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

This slide illustrates how Object-Oriented Programming maps closely to real-world concepts. A class represents a general blueprint, such as a Car class, which defines common properties and behaviours shared by all cars. These include attributes like colour, weight, speed, and model.

The methods shown, such as starting the car, changing gears, slowing down, or braking, represent actions that any car object can perform. These methods define behaviour, while the attributes define the car's state. Together, they form a complete and meaningful representation of a real-world entity.

Each individual car shown here is an object created from the same class. While they share the same structure, each object can have different attribute values, demonstrating how classes support consistency with flexibility in program design.

Object

- In Python, **everything**—numbers, strings, lists, dictionaries, etc.—is treated as an **object**.
- Objects represent **data** and provide mechanisms to interact with that data using **methods** and **attributes**.
- **Examples of Objects**
 - **Numbers**
 - 1234: An integer (**int**) **object**.
 - 3.14159: A floating-point number (**float**) **object**
 - **String:**
 - "Hello": **A string (str) object** that contains text.
 - **List:**
 - [1, 5, 7, 11, 13]: **A list (list) object** containing multiple integers.
 - **Dictionary:**
 - {"NTU": "Nanyang Technological University", "SG": "Singapore"}: **A dictionary (dict) object** mapping keys to values.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

In Python, everything is treated as an object. This includes numbers, strings, lists, and dictionaries. Each of these objects represents data and comes with built-in ways to interact with that data, which is why Python is considered a fully object-oriented language.

An object not only stores data but also provides attributes and methods that allow us to work with that data. For example, a number object supports mathematical operations, while a string object provides methods for manipulating text. This combination of data and behaviour is central to the idea of objects.

The examples shown here highlight common Python objects. Integers and floating-point numbers represent numeric data, strings store text, lists group multiple values together, and dictionaries map keys to values. All of these follow the same object-based principles, ensuring consistency across the language.

Object

• Every Object Has:

1. **A Type:**
 - Defines the **kind of data** the object represents (e.g., int, float, str, list, dict).
2. **An Internal Data Representation:**
 - **Primitive Representation:** Simple objects like int or float store data in a compact, raw form.
 - **Composite Representation:** Complex objects like list or dict combine multiple elements.
3. **Procedures for Interaction:**
 - **Objects expose methods** (functions associated with the object) for interaction.
 - Example: A string object ("Hello") has methods like `.upper()` or `.lower()`.

• An Object is an Instance of a Type

1. When you create an object, it is a **specific realization (instance)** of a broader **type (class)**.
 - Example:
 - 1234 is an **instance of the int type**.
 - "hello" is an **instance of the str type**.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted



Every object in Python has a type, which defines the kind of data the object represents and the operations it supports. This could be a simple type such as an integer or string, or a more complex type such as a list or dictionary. The type determines how the object behaves within a program.

Objects also have an internal data representation. Primitive objects, like integers and floating-point numbers, store data in a compact and efficient format. In contrast, composite objects such as lists and dictionaries combine multiple elements, allowing them to represent structured and grouped data.

Objects expose methods that enable interaction with their data. Importantly, each object is an instance of a broader type or class. For example, 1234 is an instance of the int type, while "hello" is an instance of the str type.

Object

We can,

- **create new objects** of some type

- **manipulate objects**

- **destroy objects**

explicitly using **del** just “forget” about them

python system will reclaim destroyed or inaccessible objects – called “**garbage collection**”

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted



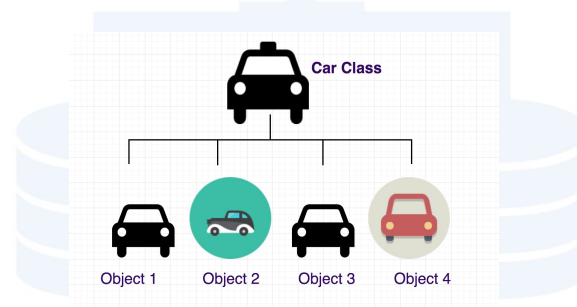
In Python, new objects are created whenever a value is assigned, a data structure is defined, or a class is instantiated. Each time this happens, a fresh object of a particular type is created in memory and can be referenced through a variable.

After creation, objects can be manipulated by accessing their attributes and calling their methods. This allows us to modify data, perform operations, and change an object’s state as the program executes. Most Python programs involve continuous interaction with objects in this way.

Objects may also be destroyed when they are no longer needed. This can occur explicitly using the `del` keyword or implicitly when objects become inaccessible. Python automatically reclaims such memory through a mechanism known as garbage collection, ensuring efficient resource management.

Class in Python

- Classes are **blueprints** for creating objects.
- A class defines **attributes (data)** and **methods (functions)**.
- Example: A Car class defines attributes (colour, speed) and methods (drive, brake). **Each car is an instance of the class.**



© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

A class in Python serves as a blueprint for creating objects. It defines a general structure that describes what data an object should store and what actions it can perform, rather than representing a single instance.

Within a class, attributes are used to store data, while methods define behaviour. For example, a Car class may specify attributes such as colour and speed, along with methods like drive and brake to describe how a car behaves.

Each object created from a class is called an instance. Although all instances follow the same class definition, they can contain different data values. This makes classes an effective way to model multiple related objects consistently.

Creating and using your own types with classes

Make a distinction between **creating a class** and **using an instance** of the class

Creating a class involves

- defining the class name
- defining class attributes and methods

for example, someone wrote code to implement a **list** class

Using the class involves

- creating new **instances of objects**
- doing operations on the instances
- *for example, $L=[1,2]$ and $\text{len}(L)$*

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

It is important to clearly distinguish between creating a class and using an instance of that class. Creating a class focuses on defining a general structure, while using a class focuses on working with specific objects created from that structure.

Creating a class involves choosing a class name and defining its attributes and methods. This is the design stage, where we describe what data the objects should hold and what operations they should support. For example, the built-in list class was implemented by someone writing its underlying code.

Using a class, on the other hand, involves creating instances and performing operations on them. For example, when we write $L = [1, 2]$ and call $\text{len}(L)$, we are using an instance of the list class.

Creating an instance of the class

```
class Employee:
    organization_name = "ABC Inc." # Class attribute

    def __init__(self, name, position):
        self.name = name # Instance attribute
        self.position = position # Instance attribute
```

class Employee: Defines a class named **Employee**.

organization_name: A class attribute shared by all instances of the class.

__init__ method: Initializes the name and position attributes when a new object is created.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted



This example shows how a class definition is used to create objects. The `Employee` class defines a general template, including a class attribute that is shared across all employee objects and an initialisation method that prepares each new instance.

The `__init__` method is called automatically when a new object is created. It assigns values to instance attributes such as `name` and `position`, ensuring that every employee object is initialised with its own specific data.

The distinction here is important. The class defines what an employee looks like in general, while each instance represents a specific employee. All instances share the class attribute, but maintain their own individual attribute values.

Creating an instance of the class

Creating instances of the Employee class

```
employee1 = Employee("Alice", "Software Engineer")
employee2 = Employee("Bob", "Product Manager")
```

When you create employee1:

- name = "Alice" and position = "Software Engineer" are passed to the `__init__` method.
- The instance employee1 is initialized with name = "Alice" and position = "Software Engineer".
- organization_name remains shared from the class.

This slide demonstrates how instances of a class are created in Python. When we write `Employee("Alice", "Software Engineer")`, we are creating a new object based on the `Employee` class and assigning it to a variable.

During this process, the values "Alice" and "Software Engineer" are passed to the `__init__` method. This method initialises the instance by assigning these values to the object's name and position attributes, making the object specific and meaningful.

Each instance stores its own attribute values, but class attributes such as organization name remain shared across all instances. This clearly illustrates the difference between instance-specific data and class-level data.

Creating an instance of the class

Creating instances of the Employee class

```
employee1 = Employee("Alice", "Software Engineer")
employee2 = Employee("Bob", "Product Manager")
```

When you create employee1:

- name = "Alice" and position = "Software Engineer" are passed to the `__init__` method.
- The instance employee1 is initialized with name = "Alice" and position = "Software Engineer".
- organization_name remains shared from the class.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted



When Python executes

```
employee1 = Employee("Alice", "Software Engineer")
```

it first allocates a new Employee object in memory. Internally, this is equivalent to calling `Employee.__new__()` to create the object, followed immediately by calling `Employee.__init__()` to initialise it.

Python then automatically calls

```
Employee.__init__(employee1, "Alice", "Software Engineer")
```

Here, `self` refers to the newly created object (which employee1 will reference). The arguments "Alice" and "Software Engineer" are passed to the parameters name and position.

Inside `__init__`, the assignments:

```
self.name = name
```

```
self.position = position
```

Create instance attributes stored in employee1's internal namespace. After this, you can access them programmatically as:

```
employee1.name    # "Alice"  
employee1.position # "Software Engineer"  
employee1.organization_name # "ABC Inc ." (from the class)
```

This clearly shows how instance attributes belong to the object, while the class attribute remains shared.

Accessing Attributes

After you have created **employee1**, **employee2** objects :

Accessing instance attributes

```
print(employee1.name)
```

Output: Alice

```
print(employee1.position)
```

Output: Software Engineer

Accessing the shared class attribute

```
print(employee1.organization_name)
```

Output: ABC Inc.

Accessing class attribute directly from the class

```
print(Employee.organization_name)
```

Output: ABC Inc.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted



Once objects such as `employee1` and `employee2` have been created, we can access their data using dot notation. When we write `employee1.name` or `employee1.position`, Python looks inside the object and retrieves the corresponding instance attributes stored during initialisation.

Instance attributes belong to individual objects. This means `employee1.name` returns "Alice", while another object may store different values. These attributes are accessed through the object reference, making it clear which instance the data belongs to.

Class attributes, on the other hand, are shared by all instances. We can access them either through an object, such as `employee1.organization_name`, or directly through the class using `Employee.organization_name`. In both cases, the same shared value is returned.

Methods in classes

What is a method?

- Procedural attribute, like a **function that works only with this class**
- Python always passes the object as the first argument
 - Convention is to use **self** as the name of the first argument of all methods
- We will include methods to display details, update the position, and calculate an annual salary in our Employee class.

A method is a procedural attribute of a class, similar to a function, but it is designed to work specifically with objects created from that class. Methods define the behaviour of objects and allow them to perform actions using their internal data.

In Python, whenever a method is called on an object, the object itself is automatically passed as the first argument. By convention, this argument is named `self`. Using `self` allows the method to access and modify the object's instance attributes.

In the Employee class, methods can be used to display employee details, update the employee's position, or calculate values such as an annual salary. These methods encapsulate behaviour that logically belongs to the employee object.

Methods in classes

```
class Employee:
    organization_name = "TechCorp Inc." # Class attribute

    def __init__(self, name, position, salary):
        self.name = name                # Instance attribute
        self.position = position        # Instance attribute
        self.salary = salary            # Instance attribute (monthly salary)

    def display_details(self):
        print(f"Employee Name: {self.name}")
        print(f"Position: {self.position}")
        print(f"Monthly Salary: ${self.salary}")
        print(f"Organization: {self.organization_name}")
```

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

In this example, the `Employee` class defines both data and behaviour. The `__init__` method initialises instance attributes such as `name`, `position`, and `salary` whenever a new `Employee` object is created. Each object therefore stores its own employee-specific information.

The `display_details` method is a class method that operates on an individual object. When this method is called, Python automatically passes the object as `self`, allowing the method to access instance attributes like `self.name` and `self.salary`.

This method also accesses the shared class attribute `organization_name`. This demonstrates how methods can combine instance-specific data with shared class-level data to present complete information about an object.

Methods in classes

```
def update_position(self, new_position):  
    self.position = new_position  
    print(f"{self.name} has been promoted to {self.position}.")  
  
def calculate_annual_salary(self):  
    annual_salary = self.salary * 12  
    return annual_salary
```

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

These two methods define behaviour associated with the Employee class. The `update_position` method modifies the object's state by updating the position instance attribute using the new value provided. Because the method operates on `self`, the change affects only the specific employee object that calls the method.

The `calculate_annual_salary` method performs a computation using the employee's monthly salary. It multiplies the instance attribute `self.salary` by 12 and returns the calculated annual salary. This method does not change the object's state but derives new information from existing data.

Together, these methods demonstrate how classes combine data and behaviour within objects.

Using the methods

Create Employee objects

```
employee1 = Employee("Alice", "Software Engineer", 5000)
employee2 = Employee("Bob", "Product Manager", 8000)
```

Display details of the employees

```
print("Employee 1 Details:")
employee1.display_details()

print("\nEmployee 2 Details:")
employee2.display_details()
```

Update position for employee1

```
employee1.update_position("Senior Software Engineer")
```

Calculate and display annual salary

```
print(f"\n{employee1.name}'s Annual Salary: ${employee1.calculate_annual_salary()}")
print(f"{employee2.name}'s Annual Salary: ${employee2.calculate_annual_salary()}")
```

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

This example shows how methods defined in a class are accessed through object instances. Two Employee objects are created using the class constructor, each storing its own name, position, and salary. Although both objects share the same class definition, they represent different real-world employees with separate data.

Next, the `display_details()` method is called on each object. The same method executes, but the output differs because it accesses the instance attributes of the object that invokes it. This demonstrates how methods work on object-specific data.

Finally, the `update_position()` and `calculate_annual_salary()` methods modify and compute values for each object. This highlights how behaviour and data are encapsulated within objects in object-oriented programming.

Object-Oriented Programming Concepts

Inheritance, polymorphism, encapsulation, and abstraction

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

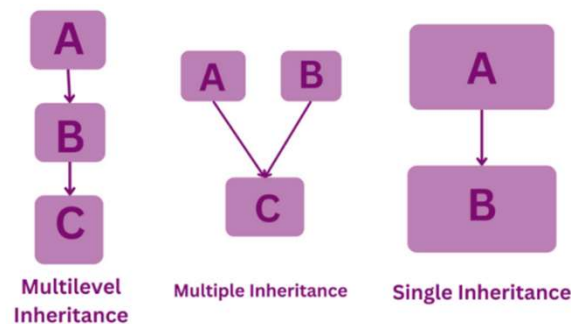
Object-oriented programming is built around four key concepts: encapsulation, abstraction, inheritance, and polymorphism. Together, these principles help structure code around real-world entities, promote reuse, improve maintainability, and enable flexible behaviour through well-defined class relationships and interfaces.

Image Source: <https://www.istockphoto.com/vector/binary-tree-gm479524481-17312880?clarity=false>

Single Inheritance

Inheritance allows a class (**child class**) to acquire properties and methods of another class (**parent class**).

Types of Inheritance



© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Single inheritance is a core concept in object-oriented programming where a child class derives from one parent class. The child class automatically gains access to the attributes and methods defined in its parent, allowing functionality to be reused instead of rewritten. This leads to clearer, more maintainable, and better-structured code.

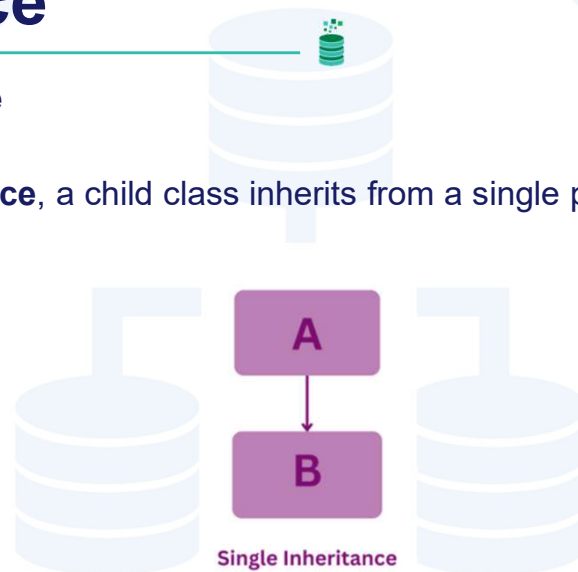
In practice, the parent class usually represents a general concept, while the child class models a more specialised version. The child can use inherited behaviour directly or extend it by adding its own attributes and methods. This approach mirrors real-world relationships in a structured and logical way.

Single inheritance also keeps class hierarchies simple and predictable. By relying on only one parent class, it avoids ambiguity and complexity, making program behaviour easier to understand, test, and debug.

Inheritance

- **Single Inheritance**

- In **single inheritance**, a child class inherits from a single parent class.



© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Single inheritance is the simplest and most commonly used form of inheritance in object-oriented programming. In this approach, a child class inherits from exactly one parent class, gaining access to its attributes and methods. This allows developers to reuse existing code rather than duplicating functionality.

Typically, the parent class represents a general concept, while the child class represents a more specific version of it. The child class can directly use the inherited behaviour or extend it by adding new features that are unique to its role. This supports a clear and logical design structure.

Single inheritance also helps maintain clarity in program design. By limiting a class to one parent, it avoids ambiguity and makes the inheritance hierarchy easier to understand, maintain, and debug.

Inheritance

Parent class

```
class Employee:
    organization_name = "TechCorp Inc." # Class attribute

    def __init__(self, name, position, salary):
        self.name = name
        self.position = position
        self.salary = salary

    def display_details(self):
        print(f"Employee Name: {self.name}")
        print(f"Position: {self.position}")
        print(f"Monthly Salary: ${self.salary}")
        print(f"Organization: {self.organization_name}")
```

Child class

```
class Manager(Employee):
    def __init__(self, name, position, salary, department):
        super().__init__(name, position, salary) # Inherit attributes from the parent class
        self.department = department # Additional attribute specific to Manager

    def display_department(self):
        print(f"{self.name} manages the {self.department} department.")
```

The **super()** keyword in Python is used to call a method from the **parent class** in a child class.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

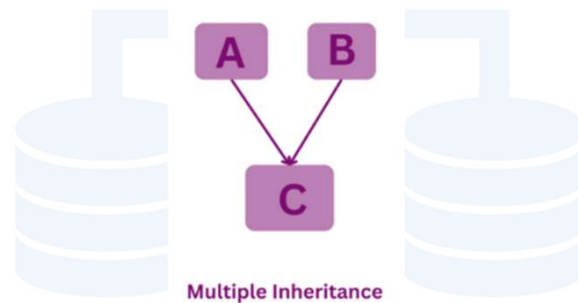
In this example, the Employee class acts as the parent class, defining common attributes such as name, position, and salary, along with a method to display these details. These attributes and behaviours represent information that is shared by all employees within the organisation.

The Manager class is a child class that inherits from Employee. Inside its constructor, the `super()` function is used to call the parent class's `__init__` method. This ensures that all inherited attributes are initialised correctly before adding the manager-specific attribute, `department`.

By using `super()`, we avoid code duplication and maintain a clean inheritance structure. The child class can extend the parent's functionality while still reusing its core logic, which is a key benefit of inheritance.

Multiple Inheritance

- **Multiple Inheritance**
- In **multiple inheritance**, a child class inherits from more than one parent class.



© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Multiple inheritance allows a child class to inherit from more than one parent class. The child class gains attributes from each parent. It can also access methods defined in both classes. This approach supports code reuse. It helps model objects with mixed responsibilities.

In the diagram, class C inherits from both class A and class B. Class C can use features from A and from B. These features do not need to be rewritten. This makes the design more flexible. It also reduces duplication in the code.

However, multiple inheritance must be used carefully. Parent classes may define methods with the same name. This can cause ambiguity. Python resolves this using a method resolution order. Clear class design helps prevent confusion and improves code maintainability.

Multiple Inheritance

```
# Parent class 1
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def display_personal_details(self):
        print(f"Name: {self.name}, Age: {self.age}")

# Parent class 2
class Employee:
    organization_name = "TechCorp Inc."

    def __init__(self, position, salary):
        self.position = position
        self.salary = salary

    def display_job_details(self):
        print(f"Position: {self.position}, Monthly Salary: ${self.salary}")

# Child class
class Manager(Person, Employee):
    def __init__(self, name, age, position, salary, department):
        Person.__init__(self, name, age) # Initialize attributes from Person
        Employee.__init__(self, position, salary) # Initialize attributes from Employee
        self.department = department

    def display_department(self):
        print(f"{self.name} manages the {self.department} department.")
```

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

In this slide, we introduce multiple inheritance. This means a child class can inherit from more than one parent class. Each parent contributes its own attributes and methods. In the example, Person stores basic personal data, such as name and age. Employee stores job-related information, such as position and salary.

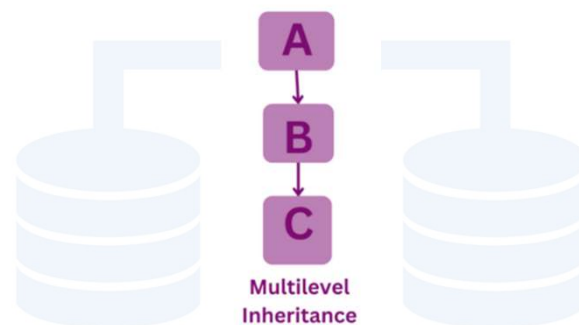
The Manager class inherits from both Person and Employee. This allows a manager object to behave as both a person and an employee. Inside the constructor, each parent's `__init__` method is called explicitly. This ensures that all inherited attributes are properly initialised.

As a result, the Manager class combines responsibilities cleanly. It adds its own department attribute. It can also access methods from both parent classes. This demonstrates how Python supports flexible class design while maintaining clear structure.

Multilevel Inheritance

- **Multilevel Inheritance**

- In **multilevel inheritance**, a child class inherits from a parent class, and that parent class inherits from another class.



© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

In this slide, we look at multilevel inheritance. This form of inheritance involves a chain of classes. One class inherits from another, and a further class inherits from that child. Each level builds on the previous one. This creates a clear hierarchy of responsibility.

In the diagram, class A is the base class. Class B inherits from A and gains its attributes and methods. Class C then inherits from B. As a result, C indirectly inherits features from both A and B.

In practice, this allows code reuse across multiple levels. Shared behaviour is defined once at higher levels. More specific behaviour is added lower down. This structure improves readability. It also supports gradual extension of functionality without duplication.

Multilevel Inheritance

```
# Grandparent class
class Organization:
    organization_name = "TechCorp Inc."

    def display_organization(self):
        print(f"Organization: {self.organization_name}")

# Parent class
class Employee(Organization):
    def __init__(self, name, position, salary):
        self.name = name
        self.position = position
        self.salary = salary

    def display_employee_details(self):
        print(f"Name: {self.name}, Position: {self.position}, Salary: ${self.salary}")

# Child class
class Manager(Employee):
    def __init__(self, name, position, salary, department):
        super().__init__(name, position, salary)
        self.department = department

    def display_manager_details(self):
        self.display_employee_details()
        print(f"Department: {self.department}")
```

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

In this example, we demonstrate multilevel inheritance using three related classes. The Organization class sits at the top. It defines shared organisational information, such as the organisation name. This information is common to all roles within the system.

The Employee class inherits from Organization. It adds employee-specific attributes, including name, position, and salary. Because of inheritance, an employee object can also access organisational details. This avoids repeating shared data and logic across classes.

Finally, the Manager class inherits from Employee. It introduces a new attribute for the department. The `super()` call ensures that all inherited attributes are initialised correctly. The manager can now display organisation details, employee details, and department information. This layered structure keeps responsibilities clear and code reusable.

Polymorphism

Polymorphism allows methods to have the **same name** but **behave differently** based on the object's context. It can be achieved through **method overriding** or **overloading**.

Types of Polymorphism

- **Compile-Time Polymorphism:**

This form of polymorphism is resolved at the time the program is compiled. It enables methods or operators with the same name to perform different tasks depending on their input parameters or context. This concept is often referred to as **method overloading** or **operator overloading**.

- **Run-Time Polymorphism:**

This type of polymorphism is determined while the program is running. It occurs when a child class provides its own implementation of a method that is already defined in the parent class. This concept is commonly known as **method overriding**.



In this slide, we introduce polymorphism. Polymorphism allows methods to share the same name but behave differently. The behaviour depends on the object that is calling the method. This makes programs more flexible and easier to extend. The same interface can support different actions.

First, consider compile-time polymorphism. Here, the method or operator behaviour is decided before the program runs. The choice depends on the number or type of input parameters. This idea is commonly known as method overloading or operator overloading. It improves readability by reusing familiar method names.

Next, we look at run-time polymorphism. This occurs while the program is executing. A child class provides its own version of a method defined in the parent class. This is called method overriding. The correct method is chosen dynamically, based on the object's type at runtime.

Compile-Time Polymorphism

Compile-Time Polymorphism: Method Overloading

- Python **does not support true method overloading** (as seen in languages like Java or C++) because it uses dynamic typing.
- However, we can achieve similar functionality using default arguments or variable-length arguments.

```
class Employee:
    def display_details(self, name=None, salary=None):
        """Method to display employee details with overloading behavior."""
        if name is not None and salary is not None:
            print(f"Employee Name: {name}, Salary: ${salary}")
        elif name is not None:
            print(f"Employee Name: {name}")
        else:
            print("No details provided.")
```

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Compile-time polymorphism describes selecting a method before execution based on the number or type of inputs. Languages such as Java and C++ support this directly through true method overloading. Python differs because it relies on dynamic typing, so method resolution does not occur at compile time.

Although Python does not provide traditional method overloading, similar behaviour can still be modelled. This is typically achieved using default parameter values or variable-length arguments, allowing one method to respond flexibly to different inputs.

In this example, the `display_details` method changes its output according to the arguments supplied. When both name and salary are provided, full details are displayed. When fewer inputs are given, the output adjusts accordingly, keeping the code simple and readable.

Run-Time Polymorphism

Run-Time Polymorphism: Method Overriding

In method overriding, **a child class redefines a method from the parent class** to provide specialized behavior.

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def work(self):
        """Parent class implementation of the work method."""
        print(f"{self.name} is performing general tasks.")
```

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Run-time polymorphism is achieved through method overriding. Here, a child class provides its own version of a method that already exists in the parent class. The key idea is that the method to be executed is chosen while the program is running, not in advance.

In this example, the Employee class defines a general work method. This represents default behaviour that applies to all employees. The method prints a generic message describing routine tasks performed by an employee.

When a child class overrides this method, Python decides which version to run based on the actual object created. If the object is a child class instance, the overridden method is executed. This allows the same method name to produce different behaviours in different contexts.

Run-Time Polymorphism

Run-Time Polymorphism: Method Overriding

In method overriding, **a child class redefines a method from the parent class** to provide specialized behavior.

```
class Manager(Employee):
    def __init__(self, name, salary, department):
        super().__init__(name, salary)
        self.department = department

    def work(self):
        """Overridden method in the Manager class."""
        print(f"{self.name} is managing the {self.department} department.")

class Developer(Employee):
    def __init__(self, name, salary, programming_language):
        super().__init__(name, salary)
        self.programming_language = programming_language

    def work(self):
        """Overridden method in the Developer class."""
        print(f"{self.name} is writing code in {self.programming_language}.")
```

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Run-time polymorphism is demonstrated here using method overriding. The Employee class defines a general structure and behaviour that is shared by all employees. It provides a common interface, including the work method, which represents generic work-related behaviour.

The Manager and Developer classes inherit from Employee and override the work method. Each child class provides its own implementation that reflects a more specialised role. A manager focuses on managing a department, while a developer focuses on writing code using a specific programming language.

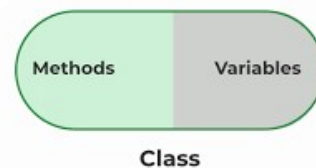
When the program runs, Python decides which work method to execute based on the actual object created. Even though the method name is the same, the behaviour differs. This is the essence of run-time polymorphism in Python.

Encapsulation

Encapsulation is the bundling of **data (attributes)** and **methods (functions)** within a class, restricting access to some components to control interactions.

Types of Encapsulation:

- **Public Members:** Accessible from anywhere.
- **Protected Members:** Accessible within the class and its subclasses.
- **Private Members:** Accessible only within the class.



© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Encapsulation focuses on grouping data and behaviour together within a class. Attributes store the data, while methods define how that data is accessed or modified. By keeping them together, a class becomes easier to understand, maintain, and reuse. This also helps prevent unintended interference with an object's internal state.

Access control is an important part of encapsulation. Public members can be accessed freely from outside the class. Protected members are intended for use within the class and its subclasses. Private members are meant to be accessed only from within the class itself, limiting direct external manipulation.

In Python, encapsulation is enforced by convention rather than strict rules. Naming patterns, such as single or double underscores, signal intended usage. This approach encourages responsible access while preserving flexibility for developers.

Encapsulation

Python achieves encapsulation by:

Using a single underscore `_` attribute to indicate a **protected attribute** (convention, not enforced).

Using a double underscore `__` attribute to make the attribute **private** (name mangling to prevent direct access)

and

Providing controlled access via methods (getters and setters).

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted



Encapsulation in Python relies mainly on conventions rather than strict access control. A single leading underscore indicates a protected attribute. It signals that the attribute is intended for internal use within the class or its subclasses, even though it can still be accessed externally.

A double leading underscore defines a private attribute. Python applies name mangling, which alters the internal attribute name. This reduces accidental access and prevents name clashes in subclasses, although it does not make the attribute completely inaccessible.

In practice, controlled access is provided through methods. Getters and setters allow safe reading and updating of attributes. This enables validation, improves maintainability, and helps preserve the consistency and integrity of an object's internal state.

Encapsulation

```
class Employee:
    def __init__(self, name, salary):
        self.__name = name # Private attribute
        self.__salary = salary # Private attribute

    # Getter for name
    def get_name(self):
        return self.__name

    # Setter for name
    def set_name(self, name):
        if isinstance(name, str) and name.strip():
            self.__name = name
        else:
            print("Invalid name. Please provide a non-empty string.")

    # Getter for salary
    def get_salary(self):
        return self.__salary

    # Setter for salary
    def set_salary(self, salary):
        if salary > 0:
            self.__salary = salary
        else:
            print("Invalid salary. Salary must be greater than 0.")
```

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

This example demonstrates encapsulation by hiding an employee's internal data and exposing it through controlled methods. The attributes `__name` and `__salary` are defined as private using double underscores. This triggers name mangling, discouraging direct access from outside the class and reducing accidental misuse.

Access to these private attributes is provided through getter methods. The `get_name` and `get_salary` functions allow values to be read safely. This ensures external code can retrieve information without directly modifying the internal state of the object.

Setter methods provide controlled updates to the data. The `set_name` method validates that the name is a non-empty string, while `set_salary` ensures the salary is positive. This validation protects data integrity and enforces meaningful constraints within the class.

Abstraction

Abstraction hides the **internal implementation** details while exposing only the necessary functionality. It helps focus on “**what to do**” rather than “**how to do it.**”

This is typically achieved using:

- **Abstract Classes:** A blueprint for other classes, containing **abstract methods** that must be implemented in derived classes.
- **Abstract Methods:** Methods declared in an abstract class but **not implemented** there. Subclasses must override them.
- Python provides the **abc** (Abstract Base Class) module to implement abstraction.

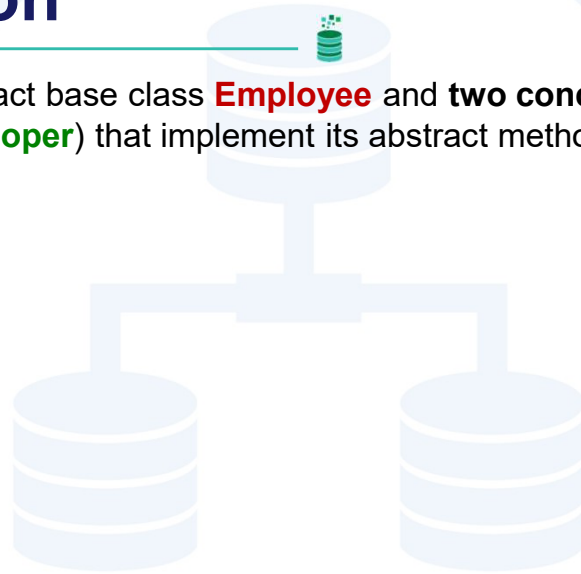
Abstraction focuses on exposing only what a user needs to know while hiding internal implementation details. It separates what a system does from how it does it. This reduces complexity and makes software easier to use and maintain. Users interact with clear interfaces without worrying about underlying logic.

In object-oriented programming, abstraction is achieved using abstract classes. An abstract class acts as a blueprint for other classes. It defines a common structure but does not provide complete behaviour. This ensures consistency across related classes.

Abstract methods are declared but not implemented in the abstract class. Subclasses must provide their own implementations. In Python, the abc module supports this approach and enforces abstraction at runtime.

Abstraction

Let's create an abstract base class **Employee** and **two concrete subclasses** (**Manager** and **Developer**) that implement its abstract methods.



© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Let us start by defining an abstract base class called Employee. This class represents a general idea of an employee rather than a specific role. It captures common behaviour that all employees are expected to have. However, it does not provide full implementations. Instead, it defines abstract methods that act as a contract for subclasses.

Next, we create two concrete subclasses: Manager and Developer. These classes inherit from the Employee abstract class. Each subclass is required to implement the abstract methods defined in Employee. This ensures consistency across different employee types while allowing role-specific behaviour.

Finally, this approach demonstrates abstraction clearly. We focus on what an employee should do, not how it is done. The base class enforces structure, while subclasses provide concrete behaviour. This design improves clarity, flexibility, and maintainability in object-oriented systems.

Abstraction

```
from abc import ABC, abstractmethod

# Abstract Base Class
class Employee(ABC):
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    @abstractmethod
    def work(self):
        """Abstract method to define the work responsibility of an employee."""
        pass

    @abstractmethod
    def calculate_bonus(self):
        """Abstract method to calculate employee's bonus."""
        pass

    def display_details(self):
        """Common method to display employee details."""
        print(f"Name: {self.name}, Salary: ${self.salary}")
```

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

This slide demonstrates abstraction in Python using an abstract base class. The ABC class from Python's abc module is used to formally define a class that cannot be instantiated on its own. The Employee class provides a common structure for all employee types. It stores shared data such as name and salary. This class describes what every employee must have, not how the work is done.

The @abstractmethod decorator marks methods that must be implemented by subclasses. Here, work() and calculate_bonus() define required behaviours. The base class does not provide implementations. This prevents creating an Employee object directly.

The display_details() method is a concrete method. It is shared by all subclasses. Each subclass, such as Manager or Developer, must provide its own logic for the abstract methods. This enforces consistent design while allowing flexibility.

Abstraction

```
# Concrete Subclass: Manager
class Manager(Employee):
    def work(self):
        print(f"{self.name} is managing the team and projects.")

    def calculate_bonus(self):
        return self.salary * 0.1 # Bonus is 10% of the salary

# Concrete Subclass: Developer
class Developer(Employee):
    def work(self):
        print(f"{self.name} is writing and debugging code.")

    def calculate_bonus(self):
        return self.salary * 0.2 # Bonus is 20% of the salary
```

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted



This slide shows how abstraction is completed using concrete subclasses. The abstract base class `Employee` defines which methods must exist. It does not describe how those methods work. Concrete subclasses such as `Manager` and `Developer` provide the actual behaviour. This converts an abstract design into executable code.

The `Manager` class implements the `work()` method to represent managerial duties. It also defines `calculate_bonus()` using a specific rule. In this case, the bonus is ten per cent of the salary. This reflects role-based behaviour in a clear way.

The `Developer` class provides its own implementations. The `work()` method focuses on writing and debugging code. The bonus calculation is higher and follows a different rule. This demonstrates how abstraction enforces structure while allowing variation in behaviour.

No part of this video shall be filmed, recorded, downloaded, reproduced, distributed, republished or transmitted in any form or by any means without written approval from the University.